

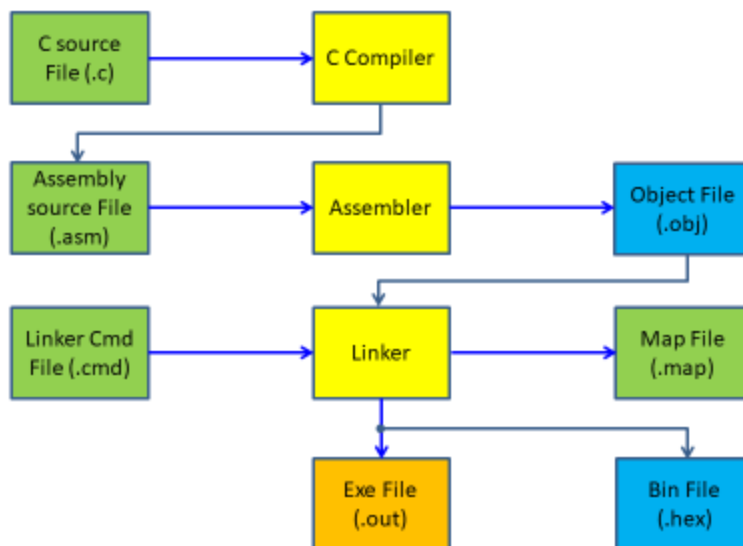
SC2107 Microprocessor System Design and Development

Optional Tutorial Question Solutions

(Tut 1, 2, 5, 6 and 7)

Tutorial #1

Q5: The diagram below shows the development toolchain of for embedded system programming.



- a. What is the function of a Linker Command File?
- A file reference by linker when assigning absolute addresses to different software sections.
 - Contains configurations of the target processor's physical memory and software sections to physical memory mapping information.
 - May include other misc linker parameters.
- b. What information is stored in the <.map> file? What is the difference between addresses stored in map file and listing file generated by the Assembler?

- Shows the absolute address information of the memory, sections and symbols defined in the program. Slide from lecture notes shown below.

Map File – Absolute Reference

- Shows the absolute address information of the memory, sections and symbols defined in the program
- Memory
 - A table showing the new memory configuration if any non default memory is specified (memory configuration).
- Sections
 - A table showing the linked addresses of each output section and the input sections that make up the output sections (section placement map).
- Global Symbols
 - Address of each global symbols
 - Symbol refers to any section, variables, structure, function name etc

CE2007 – OHL2019

42

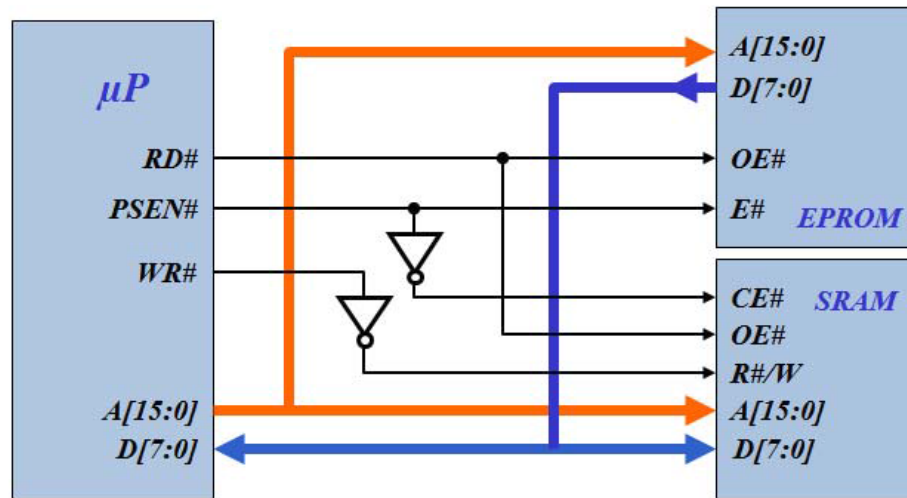
- Listing file contains symbolic/relative address references while map file contains absolute address references.
- c. What information is stored in a hex file? What is the purpose of <.hex> file?
- Hex file contains machine codes of the program, together with associated information such as the section, address of each section.
 - It is coded in ASCII text form for readability.
 - It is used typically as an input file to programmer for programming into processor or standalone system memory during production. As such, the machine code are typically partitioned into smaller chunks with CRC code to verify its integrity during programming.
- d. What is the difference between an executable file such as the <.out> file and the hex file?
- An executable contains not only the program code but information for program execution, such as entry point of the program, symbol tables, debug information etc for the processor/debugger.
 - Hex file is meant for programming purpose so contains code/data placement and error correction information.

- e. What is the difference between the <.out> file generated by the Linker and the <.obj> file generated by the Assembler? Both seem to contain the machine code that made up the user program.
- <.obj> only contains the machine code of functions in a specific file and contain symbolic/relative address information.
 - <.out> contains machine code for the entire program, together with absolute address information and other information needed to execute and debug a program (mentioned in (d) above).
- f. Can you spot any Modules/Components that is typically used when compiling a software project but is missing in the diagram?
- Listing file generated by the Assembler. Contains ASM instructions, machine code and relative addressing information within a file.

Tutorial #2

Q5: The following diagram shows the memory interface of a SRAM and an EPROM to a microprocessor that uses a memory selection signal PSEN# (Program Strobe Enable, active low) when accessing external memory. Specifically, PSEN# is asserted when the microprocessor is accessing the program code (rather than data).

- Describe the operation of the memory system with the microprocessor.
- A SRAM typically consumes higher power when its CE# is asserted (e.g. 100 μ A vs 2 μ A.) What is the disadvantage of this memory interface design and how it can be improved?



Solution:

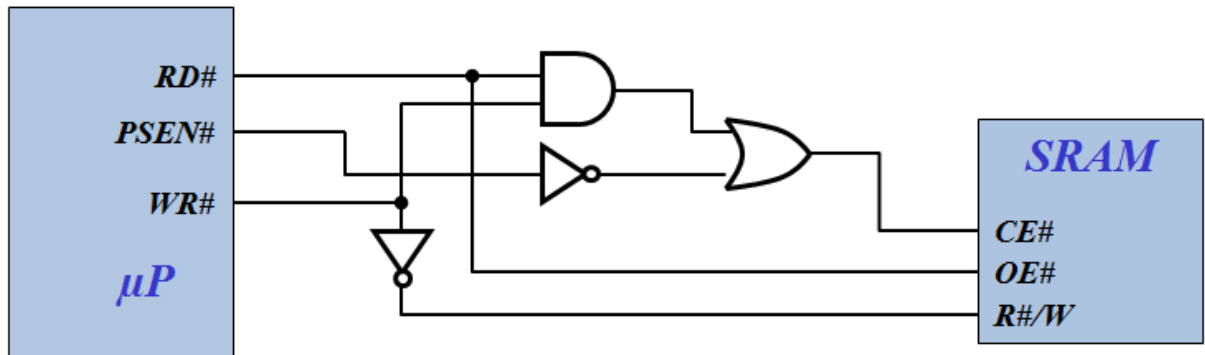
A microprocessor system needs non-volatile memory like EPROM to permanently store the program code, and uses the SRAM to store data during execution.

- The design of the system hence contains one EPROM, which is enabled when the $PSEN\#$ is asserted during fetching of the program code. By having the NOT gate between the $PSEN\#$ and $CE\#$ of SRAM, the SRAM is automatically disabled when the microprocessor is accessing the EPROM for program.

The SRAM is hence enabled whenever the EPROM is not selected. However, the SRAM will only be access when the microprocessor performs a write, or a non-EPROM read, which by default will enable the $OE\#$ or $R\#$ of the SRAM in the design.

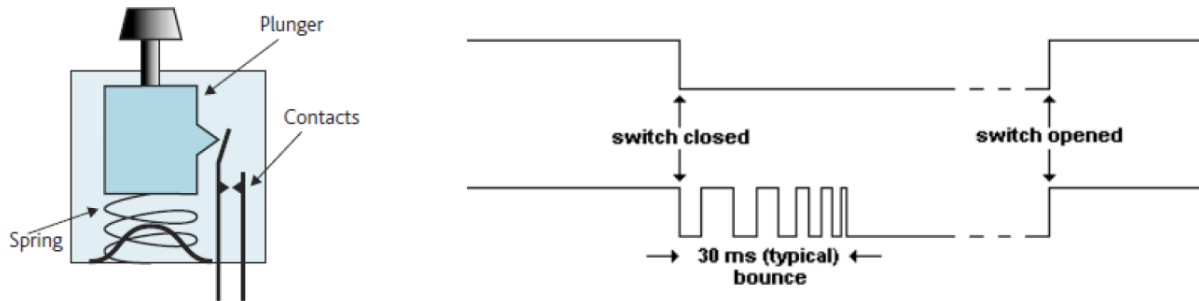
- As the SRAM is always enabled during non-EPROM operation, power is continuously consumed even when it is not accessed. To reduce the power consumption (from 100 μ A to 2 μ A), the SRAM should be disabled except during non-EPROM read/write accessed (indicated by the $PSEN\#$ signal). A more elaborate decoding scheme is hence needed to only enable the SRAM when it is accessed, such as based on the logic

$$CE\# = (PSEN\#)\# + (RD\#.WR\#)$$



Tutorial #5

Q4: Switches suffer from Switch Bounce, which produces a series of pulses when contact is made. This can be registered as a series of ON and OFF signals lasting several milliseconds instead of just the one intended single and positive switching action. The easiest software approach for debouncing is Wait and See (see pseudo code below).



```
Set Return_Value = !Switch_Pressed

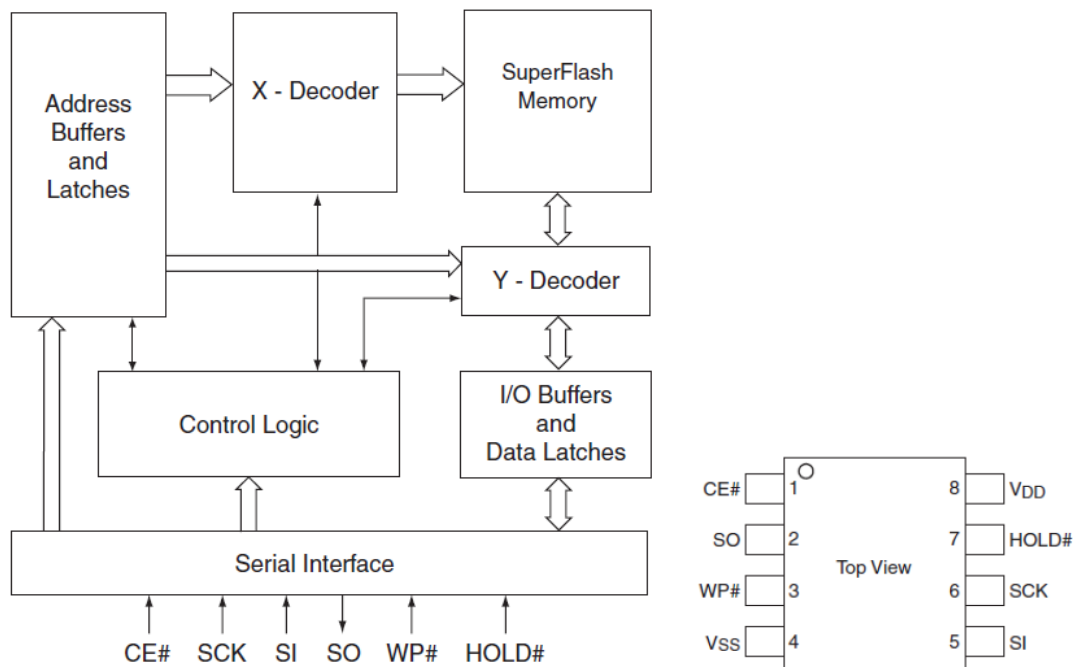
If (Switch_Pin == 0) // Switch is pressed
{
    WAIT (DEBOUNCE_PERIOD);
    If (Switch_Pin == 0) // If switch is still pressed
    {
        while (Switch_pin == 0);
        Return_Value = Switch_Pressed;
    }
}
return (Return_Value);
```

- In certain applications, a single keypress is registered only upon the release of the button. How would you modify the pseudo-code of Wait and See above, to register the key press only when the button has been released?
- Discuss a practical problem of your solution in a.

Tutorial #6

Q5: The SPI is a common interface for a microprocessor to communicate with peripheral chips. In order to communicate effectively, the programmer must have a good understanding of the protocols associated with that particular chip. These protocols may be unique to that particular chip or may be generalized due to an industry standard.

We will look at some of the protocols of the 32Mbit SPI Flash chip (SST25VF032B) to have a better understanding.

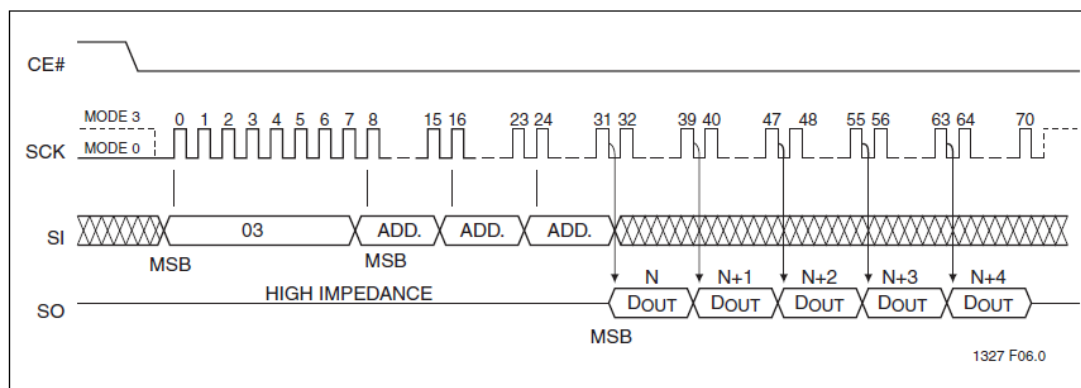


Block Diagram and Pin-Out of the SPI Flash Chip

The Device Operation Protocol is as follows:

Instruction	Description	Op Code Cycle ¹	Address Cycle(s) ²	Dummy Cycle(s)	Data Cycle(s)	Maximum Frequency
Read	Read Memory	0000 0011b (03H)	3	0	1 to ∞	25 MHz
High-Speed Read	Read Memory at higher speed	0000 1011b (0BH)	3	1	1 to ∞	80 MHz
4 KByte Sector-Erase ³	Erase 4 KByte of memory array	0010 0000b (20H)	3	0	0	80 MHz
32 KByte Block-Erase ⁴	Erase 32KByte block of memory array	0101 0010b (52H)	3	0	0	80 MHz
64 KByte Block-Erase ⁵	Erase 64 KByte block of memory array	1101 1000b (D8H)	3	0	0	80 MHz
Chip-Erase	Erase Full Memory Array	0110 0000b (60H) or 1100 0111b (C7H)	0	0	0	80 MHz
Byte-Program	To Program One Data Byte	0000 0010b (02H)	3	0	1	80 MHz
AAI-Word-Program ⁶	Auto Address Increment Programming	1010 1101b (ADH)	3	0	2 to ∞	80 MHz
RDSR ⁷	Read-Status-Register	0000 0101b (05H)	0	0	1 to ∞	80 MHz
EWSR	Enable-Write-Status-Register	0101 0000b (50H)	0	0	0	80 MHz
WRSR	Write-Status-Register	0000 0001b (01H)	0	0	1	80 MHz
WREN	Write-Enable	0000 0110b (06H)	0	0	0	80 MHz
WRDI	Write-Disable	0000 0100b (04H)	0	0	0	80 MHz
RDID ⁸	Read-ID	1001 0000b (90H) or 1010 1011b (ABH)	3	0	1 to ∞	80 MHz
JEDEC-ID	JEDEC ID read	1001 1111b (9FH)	0	0	3 to ∞	80 MHz

The following figure shows the waveform for the 'Read Sequence' command.

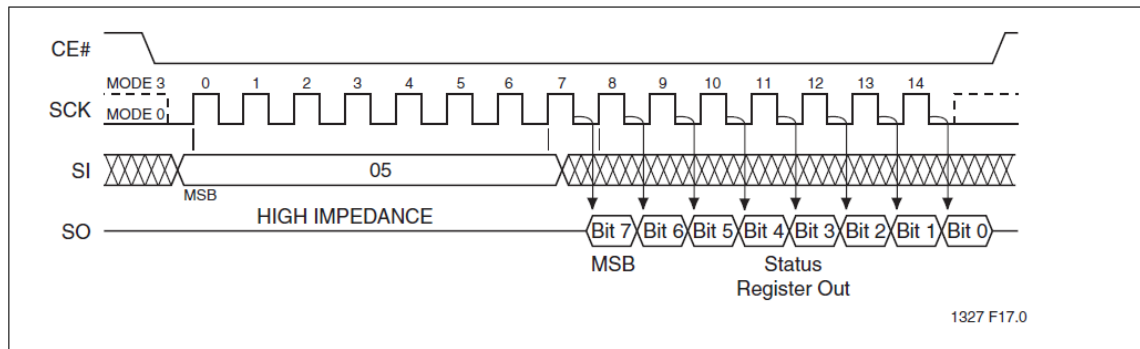


a. Decode the protocol sequence from the microprocessor's perspective.

Solution:

- The microprocessor must first select the device by driving its CE# low.
- The READ command (0x3h) is written to SI, followed by a 3-byte address (A23 – A0) that can be at any location in the device.
- The data at that address is available at the SO line.
- The device automatically increments to the next higher address after each byte of data is output.
- The data output can continue indefinitely.
- The READ command is terminated by driving CE# high at any time

The following figure shows another SPI transaction with the device.



b. Decode the protocol command.

Solution:

The command is encoded as 0x05 and according to the protocol table, it refers to Read-Status-Register (RDSR).

c. Decode the protocol sequence from the microprocessor's perspective.

Solution:

- The microprocessor must first select the device by driving its CE# low.
- The RDSR command (0x5h) is written to SI.
- The status register information is clocked out as the next byte.

Tutorial #7

Q5: Figure 3.1 and 3.2 shows the read and write access timing diagram of a SRAM. It is the signal that SRAM expects or output during a Read/Write operation. When a processor interface to the SRAM, it needs to output signals conforming to what the SRAM expect and needs to read in signals SRAM generates, all within specified timings e.g. tOE, tOHZ etc. Describe the significance of the following parameters during Read/Write operation, and whether the parameters should be a min or max value.

- tOE, tOHZ
- tWP, tDH

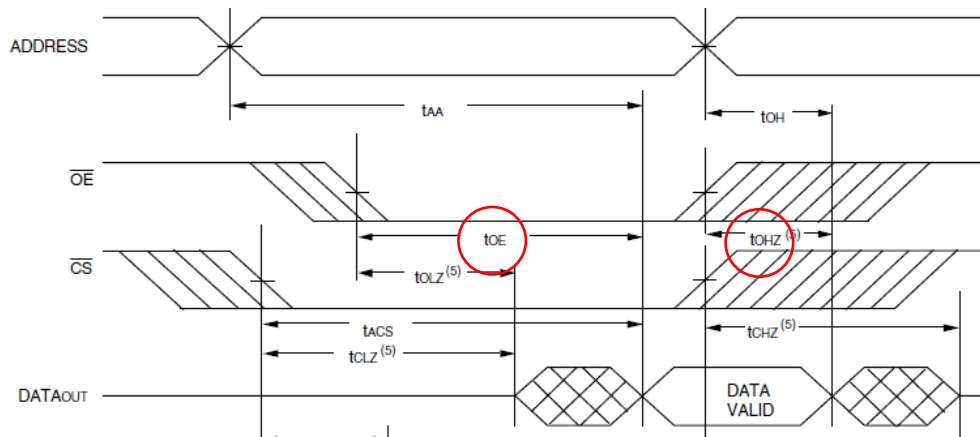


Figure 3.1: SRAM Read access timing diagram

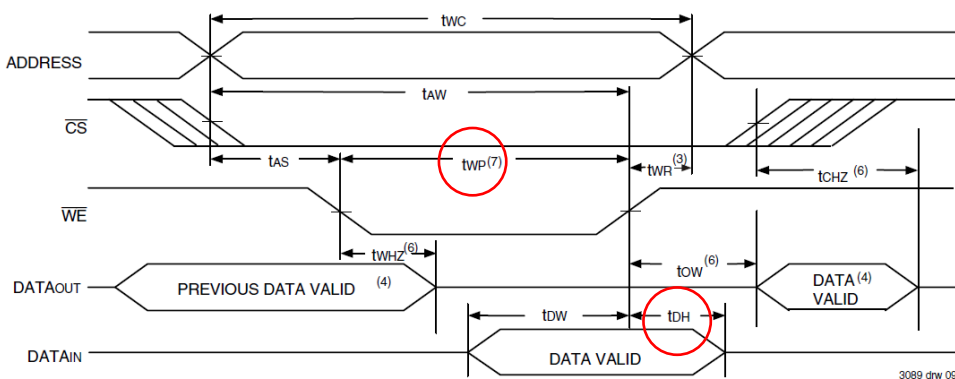


Figure 3.2: SRAM Write access timing diagram

- tOE, tOH
 - tOE is the maximum time that the SRAM needs to output the required data after its Read Strobe (OE_) is active (Logic LOW for this case).
 - tOHZ is the maximum amount of time the SRAM will hold the data on the data bus after OE_ signal is inactive.
 - The processor involved in the read operation can only perform the actual read on the data bus tOE time after asserting OE_, else the data on the bus may not be reliable.
 - Some processors require data on the data bus to be stable for a certain period after the OE_ signal is inactive. For that, ensure that the processor can support the tOHZ given in the SRAM specification, i.e. data should be read within tOHZ time after tOE is disabled.
 - tOE gives a gauge to the read access time of the SRAM, i.e. how quickly data can be read from the SRAM.

- tWP, tDH
 - tWP is the minimum time the SRAM needs after the Write Strobe (WE_) is active, in order for data on the data bus to be written into the SRAM reliably.
 - tDH is the minimum time the SRAM needs the data to be on the data bus after WE_ is inactive, in order for the SRAM to latch (read) the data reliably.
 - The processor involved in the write operation needs to
 - supply the write data on the data bus asap after asserting WE_
 - ensure WE_ is active for at least tWP time
 - hold the write data on the bus for at least tDH time after WE_ is inactive.
 - tWP and tDH give a gauge to the write access time of the SRAM.

Note that name of these signals are vendor specific and control signals for write operation may be slightly different for different vendors.

Over-provision

- over-provisioning refers to a function that secures extra space to allow for efficient use of the SSD by allo-cating a certain amount of the SSD's NAND flash to an over-provisioning space. This space can only be accessed by the SSD's con-troller and not by the host. Consisting of free blocks only, the OP region assists in efficient delivery of free blocks when wear-lev-eling or garbage collection is in progress and contributes to improved performance and lifetime of the SSD.
- Although there is no difference between the sequential and random write performance for fresh-out-of-the-box (FOB) NAND, the random write does not perform as well as the sequential write once data has been written over the entire space of the NAND. Random writes, smaller in size than sequential writes, mix valid and invalid pages within blocks, which causes frequent GC and results in decreased performance. If the OP is increased, more free space that is inaccessible by the host can be secured, and the resulting efficiency of GC contributes to improved performance. The sustained performance is improved in the same manner.

SSD Design Consideration

- **Garbage Collection**
- Technique used to free up previously written blocks
- Consolidates pages by moving and rewriting pages from multiple blocks to fill up fewer new ones.
- Old blocks are then erased to provide storage space for new incoming data.

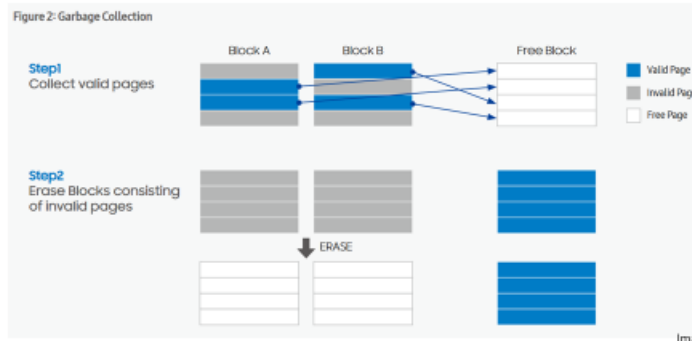


Image Source: Samsung

SSD Design Consideration

- **Write Amplification**

- Smallest unit that can be programmed is a Page
- Smallest unit that can be erased is a Block (Multiple Pages)
- To overwrite a location in Flash, it has to be erased first(unlike HDD)
- As the drive is used, data changes, and the changed data is written to other pages in the block or to new blocks. At this point, the old (stale) pages are marked as invalid and can be reclaimed by erasing the entire block. To do this, however, any still-valid information on all of the other occupied pages in the block must be moved to another block.
- The requirement to relocate valid data and then erase blocks before writing new data into the same block causes **write amplification**;
- Write Amplification happens when the total number of writes required at the flash memory is higher than the host computer originally requested.
- It also causes the SSD to perform write operations at a slower rate when it is busy moving data from blocks that need to be erased while concurrently writing new data from the host computer.

CE2107 - OHL

44

SSD Design Consideration

- **TRIM/UNMAP**

- The File Allocation Table maintained by the OS is not visible to the SSD, as such, when a file is deleted, the SSD do not know that the corresponding flash blocks can be freed. Only the storage corresponding to the FAT is modified.
- Certain Operating Systems support the TRIM/UNMAP function, which translates deleted files to the associated LBA (logical block address) on the storage device (SSD).
- The TRIM/UNMAP command notifies the drive it no longer needs data in certain LBAs (Logical Block Address) which then free up a number of NAND pages.
- The TRIM/UNMAP command needs to be supported by the OS, the drive, and the controller in order to work.
- The TRIM/UNMAP command could result in higher SSD performance from both the reduced data needed to be rewritten during garbage collection and the higher free space resulting on the drive.

CE2107 - OHL

45

SSD Design Consideration

- When Host OS delete a file, only the FAT is modified, The actual data is still intact
- The Flash Controller does not know that the deleted locations are 'invalid' from Host OS point of view.
- The controller will continue to perform Garbage Collection for these locations. Meaning unnecessary data movement which reduces reliability, endurance and write performance.
- **TRIM/UMAP** command can be sent by OS to the Flash Controller to inform it of blocks that are marked invalid so that these block can be excluded from the Garbage collection and assigned to the free pool.

Tutorial #8

Q6: What is DVFS? How does DVFS optimize the power consumption of a processor?

- DVFS = Dynamic Voltage Frequency Scaling. Varying the voltage supply to the processor core based on the frequency the CPU is running.
- Processor power is optimized since minimum voltage is supplied to enable proper execution. $\text{Power} = V \cdot I$.
- Take note of the proper sequencing requirement of changing CPU frequency vs Core Voltage levels. Details in Lecture notes.